

Homophily-based Filtration Learning For Graph Neural Networks

Anonymous Authors

Abstract

Graph neural networks (GNNs) are a powerful method of learning representations of graph-structured data. While they excel at learning class-discriminative representations of nodes in homophilous graphs, where connecting nodes tend to belong to the same class, many GNNs struggle with heterophilous graphs whose inter-class connections can muddy the message passing. Inspired by this finding, we propose a topological filtration scheme, treating graphs as 1-dimensional simplicial complexes with a filter function based on estimated edge heterophily, and introduce two methodologies that use a backbone GNN to learn from the resulting graph filtration. Both methods enhance the influence of early birth adjacent nodes in homophilous sub-graphs, yet allow for the model to learn from the full range of heterophilous and homophilous connections in the graph. Experiments show that our heterophily-filtered GNNs achieve superior node classification accuracy on both heterophilous and homophilous networks.

1 Introduction

Graphs are a natural and expressive structure to represent relationships and connections between elements, circumstances, and complex systems. Graph datasets contain two distinct types of information: the graph structure and the node features. Graph Neural Networks (GNNs) have emerged as a powerful tool for learning from graph structured data, generalizing the convolution operator to structured domains by leveraging message passing or neighborhood aggregation schemes to harness structural information. The important task of classifying nodes in networks has seen strong results thanks to GNNs. Despite their potential, GNNs face several known challenges limiting their effectiveness, one in particular being graphs with high heterophily, whose edges predominantly adjoin nodes of disparate classes.

Previous works note that heterophilous edges can muddy the message passing of traditional graph neural networks, as nodes receive messages from many neighbors that do not belong to their own class and aggregate them together with neighbors that do. As a result, many GNN methods designed for heterophilous graphs propose to rewire the graph to make it more homophilous. This often includes removing edges that are predicted in an early stage to be heterophilous. Such an approach can clarify the message passing, but does discard potentially useful topological information in the graph.

We propose a more balanced approach leveraging topological insights from persistent homology, namely persistence filtration, to construct a multilevel hierarchy of filtered versions of the graph with varying degrees of homophily. To filter our graphs, we first learn class relationships between nodes by estimating edge-level heterophily labels using the features of their adjacent nodes. We then leverage the probability of homophily assigned to each edge with persistent homology through a persistence filtration function defined over edge values to obtain a multilevel sequence of graphs well suited for hierarchical GNNs.

By combining the ability to perform message passing over a filtered higher-homophily graph with the ability to use the full topology of the original graph, our proposed approach can lead to more effective GNNs across heterophily regimes. This can be done by training only on the filtered graph and successively adding in more heterophilous edges, or by performing message passing jointly across the various filtered graph levels, using a topological attention method to weigh the various levels of the filtration as needed for the task at hand. In either case, a message-passing GNN can place greater emphasis on edges that are more likely homophilous. At the same time, even the heterophilous edges are kept at some levels of the filtration, so we retain the ability to learn from them too. This allows us to demonstrate improved node classification results on established benchmark datasets for evaluating GNN performance in low and high heterophily scenarios.

Our contributions are as follows:

- We propose a homophily-based topological filtration method for graphs, which results in a series of filtered graphs of varying levels of homophily. Since we use all the levels of the filtration, we preserve topological structure compared to approaches that drop het-

erophilous edges altogether.

- We introduce two different methods for improving backbone GNN models, especially on graphs with heterophily, by allowing them to learn from a multi-scale sequence of topological graph filtrations based on class connectivity.
- When learning from all filtration levels jointly, we use a novel filtration-based attention mechanism that allows the model to attend to more informative levels.
- We experimentally demonstrate that our methods lead to improved node classification accuracy relative to previous models.

2 Background and Related Work

2.1 Filtration Functions and Topological Filtration

We now provide the necessary background on topological filtration, how it applies to graphs, and recent advancements in learnable topological filtration schemes.

Simplicial Complexes and Topological Filtration A simplicial complex is a collection of simpler topological objects called simplices (such as points, line segments, triangles, etc.) such that every face of a simplex is also included in the simplicial complex, and the intersection of any two simplices is either empty or a shared face. Let $(K^i)_{i=0}^m$ be a sequence of simplicial complexes, or collections of topological objects such that $\emptyset = K_0 \subseteq K_1 \subseteq \dots \subseteq K_m = K$. Then, $(K^i)_{i=0}^m$ is called a filtration of K . In the filtration of K , the sequence of each simplicial complex K_i is subset to the subsequent $K_{j>i}$.

Filter Functions Let $\mathbb{E}$ be the domain of simplices and $\mathbb{K}$ the set of possible simplicial complexes over $\mathbb{E}$, the filter function $f : \mathbb{K} \times \mathbb{E} \rightarrow \mathbb{R}$ maps each simplicial complex K and simplex e to a real value $f(K, e)$. A learnable filter f_ϵ is a filter $f(\epsilon, K, e)$ is differentiable with respect to an additional parameter ϵ . It has been shown that if pairwise simplex values are distinct, such a differentiable filter function can be used in the context of k -persistent homology as a mapping of simplicial complexes that is differentiable end to end [Hofer et al., 2020].

Construction of Topological Filtration The filtration is constructed by including simplices in increasing order of their filter values. Simplices are first assigned values through a filter function or a filter function for the composition of simplices is derived. Starting with the empty set, simplices are added consecutively in sorted order by increasing the filter function value. Simplices are added such that the resulting simplicial complex contains the previous

subset simplicial complexes for lower filter function values. Each step corresponds to subcomplex K_p where p is the threshold value. For filter function $f(e)$ over simplices $e \in \mathbb{E}$ the subcomplex K_p during filtration contains all simplices with filter value less than p . The set of simplices in K_p is then $\{e \in K_p | f(e) \leq p\}$. As the filtration progresses, one can track how topological features like connected components, loops, and voids appear and disappear. These features are summarized using tools like persistent homology.

Having now described the relevant concepts in computational topology abstractly, we give some background on graphs and the application of computational topology to graphs.

Graphs and Homophily Let $G = (\mathcal{N}, \mathcal{E})$ be a graph consisting of node set $v \in \mathcal{N}$ and edge set $\{e = (u, v) | e \in \mathcal{E} \text{ and } u, v \in \mathcal{N}\}$ where edge $e = (u, v)$ is said to adjoin nodes u and v . We denote a neighborhood around node v as $N(v) = \{u \in \mathcal{N} | (u, v) \in \mathcal{E}\}$ to consist of all nodes in $\mathcal{N}$ that share an incident edge with v in $\mathcal{E}$. For $k \geq 1$ we denote the k -hop neighborhood of node v , $N_k(v)$, as the set of all nodes $u \in \mathcal{N}$ such that there is a connected path of k or fewer edges adjoining u and v . For nodes u and v with labels y_u and y_v and edge set $\mathcal{E}_i$ for graph G_i in the filtration sequence, the homophily ratio [Zhu et al., 2020] is given by $h_i = \frac{1}{|\mathcal{E}_i|} \sum_{v \in \mathcal{E}_i} \frac{|\{u \in \mathcal{N}(v) : y_u = y_v\}|}{|\mathcal{N}(v)|}$.

Topological Filtration of Graphs We can interpret a graph G as a 1-dimensional simplicial complex whose vertices (0-simplices) are the graph nodes and whose edges (1-simplices) are the edges. In the topological filtration of graphs, we can think of filtration as a growth process of a weighted graph. As the graph grows, new edges and nodes are introduced, introducing new connected components and neighborhood structures.

Node neighborhoods thus share this property: for each node $v_i \in \mathbf{V}_{p_i} \cap \mathbf{V}_{p_j}$, e.g. $\mathcal{N}_{p_i}(v_i)$ for $v_i \in \mathbf{V}_{p_i}$ and $\mathcal{N}_{p_j}(v_i)$ for $v_i \in \mathbf{V}_{p_j}$, we have that $\mathcal{N}_{p_i}(v_i) \subseteq \mathcal{N}_{p_j}(v_i)$. We give all graphs the full node set found at the lowest threshold level of the filtration sequence, but nodes that would not otherwise exist in the graph at a higher persistence level are disconnected. Early birth nodes, which appear for lower value thresholds, then remain in subsequent graphs in the filtration sequence, with neighborhood structure for each node potentially differing for each node between different graphs in the sequence.

3 Methodologies for Filtration Learning

Our method has three steps. First, we train an edge classifier to predict the probability that each edge in the graph is homophilous. Second, we use these edge homophily scores to filter the graph, allowing us to construct versions of the

graph with several levels of filtration. Third, we perform message passing over each level of the filtration, either in successive stages of training or jointly all at once.

3.1 Estimating Edge Homophily Scores

We first filter graphs by the likelihood an edge adjoins nodes of disparate classes. To accomplish this we first frame the task of node classification as an edge classification problem, where the edge classifier serves as a filter function that prescribes filter values to edges representing the inferred likelihood that an edge is homophilous.

Given a graph $G = (\mathcal{N}, \mathcal{E})$ containing labeled nodes v with labels y_v and node features h_u and h_v for a subset or all of node set $\mathcal{N}$, we can derive binary edge labels for edge $e = (u, v)$ as $y_{(u,v)} = \begin{cases} 1 & \text{if } y_u = y_v \\ 0 & \text{if } y_u \neq y_v \end{cases}$ for graphs with binary or multi-class labels. Our edge filter function $f(e)$ is a multilayer perceptron with a single logistic output defined as $\text{MLP}(\text{COMBINE}(h_u, h_v))$. In our where h_u and h_v are the feature vectors of adjacent nodes u and v and COMBINE is implemented via concatenation (other choices of functional operator, such as dot product, are possible).

We then train the filter function f using the derived homophily edge labels $y_{u,v}$ on a subset of the graph and infer values between 0 and 1 to all remaining edges of the graph, predicting whether an edge is heterophilous or not (higher scores meaning a higher probability of heterophily).

3.2 Homophily-based Topological Filtration of Graphs

We perform filtration on graphs using a sorted ordering of 1-simplices, or edges, with filter values assigned by $f(e)$ corresponding to the likelihood that an edge is heterophilous. At the i -th level of our filtration, we use a filter threshold p_i , and then add edge $e = (u, v)$ and its incident nodes u and v if $f(e) \leq p_i$ and such that the expanded graph G_i at level i of the filtration sequence remains a simplicial complex. We observe the topological filtration sequence at P levels of filtration thresholds to model multiscale topological information. A smaller threshold value produces a more highly filtered graph with fewer edges that are more likely to be homophilous. Thus, we obtain a hierarchy of graphs $G_1, \dots, G_P$ where for $i \in [1, \dots, P]$, $G_{i-1} \subseteq G_i$; that is, G_{i-1} is an induced subgraph defined on a subset of the vertices in G_i . The last level of the filtration is always the (unfiltered) input graph. In contrast to hierarchical graph-level representation learning methods that learn to pool each input graph [Ying et al., 2018], this gives us a multiscale hierarchy of graph representations of the underlying data to use as input to a graph neural network.

3.3 Graph Neural Networks with Topological Filtration

We consider two methods for learning node representations using GNNs, which exploit the topological information from the nested sequence of graphs obtained through topological filtration, referred to as the graph hierarchy, in the context of training GNNs. The first method divides the training of a GNN across graph hierarchies, while the second modifies the architecture to pass messages jointly between all hierarchical graph levels.

3.3.1 Multi-Scale Successive Training

Our first method does not require modifying the architecture of a graph neural network; rather, we divide its training stages across different levels of the filtration. Training begins on a more heavily filtered version of the graph with fewer edges that our edge classifier estimates to be more likely homophilous. However, in contrast to previous work [Huang et al., 2022], which stops after training on a graph with heterophilous edges pruned, we propose to continue training for additional epochs on successively less filtered versions of the graph. That is, instead of training a GNN for N epochs on the finest graph G_P , we train it for $\frac{N}{P}$ epochs each on graphs $G_1, \dots, G_P$, using the embeddings from G_{i-1} to initialize the embeddings of corresponding nodes in $G_i \forall i \in [1, \dots, P]$. Training begins with the smallest (most highly filtered) graph and iteratively increases in graph size.

The MLPs and node embedding weight matrices for target and neighboring node embeddings are shared between graph hierarchies. In effect, this allows us to initialize the GNN training at each successive layer of the hierarchy with the results of training on a more homophilous previous layer. Prior work has shown that initializing GNNs with MLPs produces a loss landscape with larger low-loss regions, making it easier for a model to learn an effective solution [Han et al., 2023]. Likewise, we expect that an improved initialization will allow a GNN to learn more effectively in a heterophilous graph. For shallow network embedding methods, it has also been found that node embeddings learned in a coarse version of a graph can serve as effective initializations for training on the original (fine-granularity) graph [Chen et al., 2018]. By designing a heterophily-specific filtration rather than a general graph coarsening measure used in [Chen et al., 2018], we show that this insight has particular utility for learning on heterophilous graphs.

As the GNN architecture is not modified, this method can be used with any backbone GNN. In this work, we use a GraphSAGE architecture which learns node features via message passing in the spatial domain [Hamilton et al., 2017]. This is a conventional GNN model that was shown to have some desirable properties for

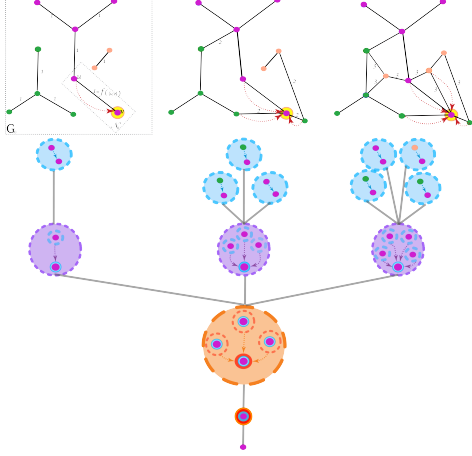


Figure 1: A sequence of three nested graphs from topological filtration (top row) offering three levels of graph resolution. Messages are shown being passed (dotted arrow) to the bottom right node that, having survived the filtration, is common to all graphs in the multi-scale sequence, differing in neighborhood connectivity. For each subgraph G_p of the total P graphs in the graph hierarchy $\{G_0, \dots, G_p, \dots, G_P\}$, messages are passed from each neighborhood $N^r(v)$ to the target node. Messages at each graph level are then aggregated within neighborhoods (highlighted in blue). Messages from all neighborhoods within each graph are then aggregated between one another (highlighted in purple). We introduce (highlighted in orange) across neighborhood aggregation, where messages are passed across neighborhoods in the multi-scale sequence of graphs. The target node’s feature representation is then updated with the aggregated embeddings.

learning from graphs with heterophily [Zhu et al., 2020], though it is not a heterophily-specific model and thus allows us to see improvement from using our graph filtration.

3.3.2 Multi-Scale Joint Training

Our second option extends the message passing framework of a graph neural network across multiple hierarchical levels jointly. Message passing occurs during a single layer and iteration k where each node’s feature h_v^k is updated to incorporate the features associated with nodes within its neighborhood and the layer’s learnable parameters denoted ε and W . In Fig. 1, node v is highlighted in yellow within each graph of the nested filtration sequence and the dotted blue arrows denote the messages passed to node v in each neighborhood $N_p^r(v)$ where p denotes the graph level in the sequence at filtration value $p \in \{0, \dots, P\}$ that the neighborhood being considered corresponds to. A single node that exists within multiple graphs of the filtration sequence can have neighborhood structures that differ but are subsets to subsequent neighborhoods later in the sequence of

graphs resulting from the filtration.

Aggregation Within Neighborhoods Messages are then aggregated for each neighborhood $N_p(v)$ across nodes in the target node v ’s neighborhood at filtration level given by threshold p . In Figure 1 this is illustrated with the regions highlighted in purple.

$$h_{v,p} = \text{AGG}_{u \in N_p(v)} \{h_{u,p}\} \quad (1)$$

Here AGG can be any operation producing a single vector from a multiset of vectors. Following [Hamilton et al., 2017], we use mean-pooling for this and subsequent AGG steps, counting each node as its own neighbor.

Aggregation Across Filtration Levels This work introduces the step highlighted in orange in Figure 1. Namely, once messages from each neighborhood have been aggregated, messages are then aggregated across all target node neighborhoods in all graphs within the sequence of graphs to which target node v belongs.

$$h_v = \text{AGG}(\{h_{v,p}\}) \quad (2)$$

In Figure 1 this is illustrated with the regions highlighted in orange.

Having updated node feature representations with information within and across graph levels, we perform one more round of message passing at each level p , modeling a node’s input features separately from the neighbor features [Zhu et al., 2020]. For node v_i belonging to node set V_{p_i} of subgraph G_{p_i} , the feature representation $h_{v_i,p}^k$ at GNN layer $k \in \{1, \dots, K\}$ first combines self-representations from the previous level of aggregation with aggregated neighbor information:

$$h_{v,p}^k = \text{COMBINE}(W_s^{k-1} h_v^{k-1}, \text{AGG}_{u \in N_p(v)} h_u) \quad (3)$$

Once this is done, the resulting embeddings are combined across persistence subgraphs. As an example for two persistence levels p_i and p_j , we have $h_v^k = \sigma(\text{COMBINE}(h_{v,p_i}^k, h_{v,p_j}^k))$, where σ is an activation function (we use GELU).

Learned Node Filtrations of Graphs

This work introduces a second, learnable node filter function for learning a filtration of nodes in graphs within the graph hierarchy during training. Filtration of nodes in the graph hierarchy aims to learn which graphs (and their respective nodes) in the nested graph sequence obtained from topological filtration are most informative for multi-scale or hierarchical graph learning. Graph Isomorphism Networks [Xu et al., 2018], (GIN- ε) not only have theoretical expressivity benefits for distinguishing graph substructures but have also been shown to be a viable candidate as a

learnable filter function for persistent homology when also combined with a multilayer perceptron [Hofer et al., 2020]. We then use a learnable node filter function during Multi-Scale Joint Training defined as

$$f_p(\mathcal{N}_p, u) := \text{MLP}(\text{GIN}_\varepsilon^p(h_u^p)) \rightarrow \mathbb{R}$$

where h_u^p is the embedding representation of node u for graph level p of the graph hierarchy. GIN_ε^p is a GIN network which is learnable and differentiable in ε [Xu et al., 2018] which we use to learn node representations for graph level p , and MLP is a multilayer perceptron which takes the resulting embedding from GIN_ε^p . The resulting real-valued output of f_p is the learned filter value, which is a scalar $q_p(u)$ for node u in graph level G_p .

This work introduces a methodology for using the learned filter values $q_p(u)$ to filter graphs within the graph hierarchy. To learn the importance and reduce or increase the contribution of graphs in the graph hierarchy during multi-scale joint aggregation, we use the q_p filter value for attention type based aggregation by weighing the aggregated embedding obtained from graph G_p in the hierarchy before multi-scale aggregation across all graphs G_i in the multi-scale filtration sequence of P graphs $G_0, G_1, \dots, G_P$. Before the combination step of multi-scale aggregation, we then factor each node’s resulting embedding from each graph level by its respective filter value q_p . In doing so, the learned filter value can learn to eliminate or accentuate the contribution of embeddings from specific graphs in the multi-scale graph sequence.

The learned filtration of graphs in the graph hierarchy during multi-scale joint aggregation can then be expressed as the combined embedding from each subgraph, where each subgraph’s embedding is obtained with a backbone GNN and a filter function $f_p(\mathcal{N}_p, u)$, unique to each graph level, for a final embedding that is weighted by a filter value q_p before global combination. For node v_P in the highest resolution graph G_P in the filtration sequence, we then have:

$$h_v = \text{AGG}(\{q_p h_{v,p}\}) \quad (4)$$

This generalizes Equation 2 to use the learned filtration weights as attention scores when combining messages from different filtration levels.

From the original precomputed filtration sequence of graphs, we then learn a subset filtration sequence of graphs through the learnable node filter function $f_p(\mathcal{N}_p, u)$, which learns a weighted attention on each node in each graph level to obtain a learned filtration sequence of graphs.

4 Experiments

Implementation We implement all models with PyTorch and all GNNs with Pytorch Geometric [Paszke et al., 2019, Fey and Lenssen, 2019]. Topological filtration is performed with Dionysus [Morozov, 2007].

Hyperparameters and Computing Environment We employ early stopping based on validation scores (computed every 8 epochs) for a deviation of $1e - 3$, with patience of 4 rounds of validation accuracy scores for the edge filter function’s training and 10 for the multi-scale model used. For dropout, the edge filter function uses a value of 0.3, and multi-scale models have a value of 0.65. As the size of the datasets tested varied, hidden dimensions varied to account for the limitations of the machine being used and model overfitting. We note that overfitting is a challenge on many common heterophilous graph benchmarks [Zhu et al., 2023], and experienced this especially with our larger joint message passing models, leading us to use aggressive dropout to combat this. We perform a parameter sweep for each of the following hyperparameters over the values provided in Table 1

All experiments were run on a laptop with 3 GB GeForce GTX 970M with 1280 CUDA Cores GPU and 3.5GHz i7-6700HQ processor running Ubuntu, Linux.

4.1 Datasets

We use several different standard node classification benchmarks covering a range of homophily levels.

Planetoid (CiteSeer, Cora, and PubMed): These are widely used citation networks, where nodes represent documents, and edges represent citation links between them. The task is typically node classification, where the goal is to predict the category of each document [Yang et al., 2016]. These datasets are well known to have high homophily.

Wikipedia Network (Chameleon): is a graph derived from the page-to-page link network of Wikipedia articles. Nodes represent Wikipedia pages, and edges represent hyperlinks between them. The nodes are classified based on the traffic levels of the web pages. This dataset is used to study the performance of graph learning methods designed for heterophilous networks. [Rozemberczki et al., 2021].

WebKB (Cornell, Wisconsin, and Texas): The WebKB datasets, including Cornell, Wisconsin, and Texas, consist of webpages collected from computer science departments of various universities. Nodes represent webpages, and edges represent hyperlinks between them. The task is to classify the type of webpage (e.g., course, faculty, student) and these datasets are also known to have high levels of heterophily [Peixoto and Emmert-Streib, 2019].

Parameter	Values
Filtration thresholds (in addition to input graph)	(0.5,0.7,0.9), (0.8,0.9), (0.6,0.9), (0.7), (0.8), (0.9), (0.1)
Learning Rate of Multi-Scale GNN (MsST or MJT)	3e-4, 3e-3, 3e-2, 1e-4, 1e-3, 1e-2
Learning rate of edge filter function	1e-3, 3e-3, 1e-2, 3e-2
Weight decay	5e-8, 5e-6, 5e-5, 5e-4, 0

Table 1: Values considered for different hyperparameters

Dataset	Subdataset	Nodes	Edges	Classes
Planetoid	Cora	2,708	5,429	7
	CiteSeer	3,327	4,732	6
	PubMed	19,717	44,338	3
WikipediaNetwork	Chameleon	2,277	31,421	5
WebKB	Cornell	183	295	5
	Wisconsin	251	499	5
	Texas	183	309	5

Table 2: Summary of datasets and their characteristics.

Node Classification Results In Table 3, we report the test accuracy results of MsST and MsJT in the context of the reported accuracies of other state-of-the-art models with equivalently proportioned train, validation and test splits (we use 60%, 20%, and 20% respectively, following [Pei et al., 2020] and the subsequent works we compare against)¹.

First of all, we note that our methods excel in all except one dataset, achieving the highest and second highest accuracies. This indicates the potential of our hierarchical filtration to benefit GNNs across the homophily spectrum.

Second of all, while the improvement is modest on the high-homophily datasets Pubmed and Cora (though we do observe a larger percentage improvement on Citeseer), we see a pronounced improvement on several of the high-heterophily datasets. For instance, MsST is over 10% more accurate than the best baseline considered on the Texas dataset, and close to 10% on the Cornell dataset (these are the two datasets with the lowest homophily ratios). This may suggest that indeed our homophily-based filtration is effective especially in high heterophily graphs, where at some levels the filtration uncovers high-homophily subgraphs for the model to learn from.

Finally, we note that on all datasets, our model outperforms SA-SGC, which simply uses the edge classifier to remove heterophilous edges from the graph. This may suggest that by keeping access to them for our model, but simply only at some filtration levels, we can achieve superior performance. Heterophilous edges are not necessarily just noise.

4.2 Limitations

We also investigated the use of additional heterophily benchmarks proposed in [Platonov et al., 2023]. Our models tended to do less well on them, which we believe is because these datasets tend to have low label informativeness (feature correlation with label). This degrades the performance of our feature-based edge heterophily predictor used to construct the edge classifier, and is a limitation shared by previous works that also use an edge classifier to preprocess a graph before performing message passing [Huang et al., 2022, Xue et al., 2024]. To extend our methods to such benchmarks, we may need to develop an alternative heuristic to predicting edge homophily scores, perhaps involving a simpler GNN model rather than a feature-based MLP.

5 Related Work

Here, we review related works on graph neural networks, whether using similar techniques based on filtration learning, or focused on the problem we consider of learning from graphs with heterophily.

5.1 Graph Neural Networks for Heterophilous Graphs

For a recent overview of graph neural networks designed to handle graphs with high heterophily, see [Zhu et al., 2023, Luan et al., 2024]. Methods designed to handle graphs with heterophily usually focus on one of two challenges posed by heterophily. The first is that in heterophilous graphs, nodes may not have relevant neighbors (e.g. neighbors of the same class) to attend to, and one solution is to expand the receptive field of graph neural networks to allow them to learn non-local dependencies. Such approaches include higher-order neighborhood aggregation, proposed by [Zhu et al., 2020], rewiring the graph based

¹For Geom-GCN, we report results from the original version of the paper: https://openreview.net/attachment?id=S1e2agrFvS&name=original_pdf.

Model	Texas	Wisconsin	Chameleon	Cornell	CiteSeer	Pubmed	Cora
Hom. Ratio	0.06	0.16	0.25	0.11	0.74	0.80	0.83
MsST	96.72%	95.62%	90.12%	95.63%	88.73%	71.04%	93.17%
MsJT	89.27%	93.57%	91.25%	91.71%	93.49%	91.78%	95.39%
Geom-GCN [Pei et al., 2020]	75.67%	76.47%	89.32%	66.66%	79.12%	90.11%	90.57%
H ₂ GCN [Zhu et al., 2020]	84.86%	86.67%	59.39%	82.16%	77.07%	89.59%	87.81%
SA-SGC [Huang et al., 2022]	83.52%	84.52%	64.94%	81.12%	76.88%	87.55%	87.36%
DHGR [Bi et al., 2024]	85.68%	85.01%	74.57%	82.88%	71.96%	80.09%	83.93%
NSD [Bodnar et al., 2022]*	85.95%	89.41%	68.68%	86.49%	77.14%	89.49%	87.30%
DCM [Battiloro et al., 2023]	85.71%	89.33%	53.76%	N/A	78.60%	88.61%	86.63%

Table 3: Test accuracy results for node classification over datasets, sorted by homophily level, for our methodologies (MsST and MsJT), heterophily oriented model designs, and topological deep learning (TDL) models (last two rows). For our baselines, we report the results of the best-performing variant from the original paper. The highest accuracy scores are highlighted in green, with the second highest outlined in blue. We find that our methods outperform the baselines on almost all datasets across the homophily spectrum.

on other forms of node similarity [Pei et al., 2020], or introducing non-local attention [Zhu et al., 2020]. Such approaches may incur a higher computational cost.

Our work falls into a second category of approaches, which recognize that heterophily also may muddy the message passing of graph neural networks by causing messages to be aggregated from irrelevant neighbors. Such approaches modify the message passing to better suit heterophilous graphs, for example by performing it over signed [Yan et al., 2022] or directed [Rossi et al., 2023] edges, or by learning a class compatibility matrix that may be used to control message passing [Zhu et al., 2021]. Other approaches include rewiring the graph to be better suited to heterophily, by connecting nodes based on some other measure of similarity or distance [Pei et al., 2020, Bi et al., 2024, Li et al., 2023].

Most related to our work, this rewiring has been performed with an edge classifier that predicts if an edge is heterophilous or homophilous [Huang et al., 2022, Xue et al., 2024]. A backbone graph neural network is trained on a graph that has been preprocessed using the classifier to improve its homophily properties. The edge classifier may be used to filter the graph only by pruning heterophilous edges [Huang et al., 2022], or new edges may also be added that are predicted to be homophilous [Xue et al., 2024]. Both methods result in a single output graph which loses any information that the heterophilous edges may contain. In contrast, our approach gives a model access to the input graph at a variety of filtration levels, giving us the best of both worlds: an ability to use all the information in the original graph, while also being able to emphasize the high-homophily connections in a more filtered graph.

5.2 Topological Data Analysis for Graph Neural Networks

Graph filtration learning using persistent homology has been used for graph-level representation learning [Hofer et al., 2020]. For node classification, persistent homology has also been used to guide message passing in graph neural networks [Zhao et al., 2020] for semi-supervised node classification. More related to our work, [Leventhal et al., 2023] has used graph neural networks to classify topological objects in image data, learning successively or jointly from hierarchical levels of a topological filtration. Here, we show how to derive such a hierarchical filtration for graphs specifically based on homophily, and we develop a node-level filtration that allows us to attend to different levels of filtered graphs.

While our work focuses on persistent homology, it is possible to use other topological concepts to design neural networks for graphs. Neural sheaf diffusion [Bodnar et al., 2022] is another topologically-inspired technique to improve the performance of graph neural networks on heterophily, built on a different set of learned topological structures called sheaves and designing a sheaf convolution operation analogous to graph convolution. A Differentiable Cell Complex Module has also been proposed to learn a cell complex and perform message passing over the resulting cell complex, resulting in a tradeoff between leveraging the expressivity of the resulting topological structure and maintaining computational tractability [Battiloro et al., 2023].

6 Conclusion

We propose graph neural network methods that learn from hierarchies of graphs representing a sequence of nested graphs obtained from topological filtration. Treating graphs from the perspective of 1-D simplicial complexes,

our filtration is defined as being over 1-cells (graph edges). We obtain a sequence of simplicial complexes, where each complex in the series is subset to the subsequent, from a sorted ordering of 1-cells, thresholded by filtration values prescribed to simplexes through a filter function defined as a learning model trained to infer if edges are homophilous, akin to a class similarity measure between 0-cells (graph nodes). The proposed approach is generalizable to any edge based classification task or graph with weighted edges that afford an ordering. We then present two methods for hierarchical graph learning. The former, achievable for any standard GNN model, performs message passing across subsequent graphs in the graph hierarchy, aggregating node embeddings sequentially moving up the graph filtration sequence. the ladder introduces a novel multi-scale message passing scheme with aggregation performed jointly across graphs in the filtration sequence, with learned attention based aggregation or learned topological filtration of subset graph nodes, effectively learning the degree of contribution graphs within the hierarchy contribute to learned aggregated node embeddings. Our results demonstrate promise for increased node classification accuracy on heterophilous networks compared to conventional GNNs.

References

- [Battiloro et al., 2023] Battiloro, C., Spinelli, I., Telyatnikov, L., Bronstein, M., Scardapane, S., and Di Lorenzo, P. (2023). From latent graph to latent topology inference: Differentiable cell complex module. *arXiv preprint arXiv:2305.16174*.
- [Bi et al., 2024] Bi, W., Du, L., Fu, Q., Wang, Y., Han, S., and Zhang, D. (2024). Make heterophilic graphs better fit gnn: A graph rewiring approach. *IEEE Transactions on Knowledge and Data Engineering*.
- [Bodnar et al., 2022] Bodnar, C., Vasilache, A. T., Rüttgers, M., Afshar, M., Bronstein, M., Böhm, M., and Monti, F. (2022). Neural sheaf diffusion: A topological perspective on heterophily and oversmoothing in gnns. *arXiv preprint arXiv:2202.04579*.
- [Chen et al., 2018] Chen, H., Perozzi, B., Hu, Y., and Skiena, S. (2018). Harp: Hierarchical representation learning for networks. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32.
- [Fey and Lenssen, 2019] Fey, M. and Lenssen, J. E. (2019). Fast graph representation learning with pytorch geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*.
- [Hamilton et al., 2017] Hamilton, W. L., Ying, R., and Leskovec, J. (2017). Inductive representation learning on large graphs. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, pages 1025–1035.
- [Han et al., 2023] Han, X., Zhao, T., Liu, Y., Hu, X., and Shah, N. (2023). Mlpinit: Embarrassingly simple gnn training acceleration with mlp initialization. In *ICLR*.
- [Hofer et al., 2020] Hofer, C., Graf, F., Rieck, B., Nießhammer, M., and Kwitt, R. (2020). Graph filtration learning. In *International Conference on Machine Learning*, pages 4314–4323. PMLR.
- [Huang et al., 2022] Huang, M., Cai, C., Liang, Y., Wang, J., and Song, L. (2022). Revisiting the role of heterophily in graph representation learning: An edge classification perspective. In *Advances in Neural Information Processing Systems*.
- [Leventhal et al., 2023] Leventhal, S., Gyulassy, A., Pascucci, V., and Heimann, M. (2023). Modeling hierarchical topological structure in scientific images with graph neural networks. In *2023 IEEE International Conference on Image Processing (ICIP)*, pages 2995–2999. IEEE.
- [Li et al., 2023] Li, S., Kim, D., and Wang, Q. (2023). Restructuring graph for higher homophily via adaptive spectral clustering. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 8622–8630.
- [Luan et al., 2024] Luan, S., Hua, C., Lu, Q., Ma, L., Wu, L., Wang, X., Xu, M., Chang, X.-W., Precup, D., Ying, R., et al. (2024). The heterophilic graph learning handbook: Benchmarks, models, theoretical analysis, applications and challenges. *arXiv preprint arXiv:2407.09618*.
- [Morozov, 2007] Morozov, D. (2007). Dionysus - a library for computing persistent homology. <http://www.mrzv.org/software/dionysus/>. Accessed: 2024-08-10.
- [Paszke et al., 2019] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. (2019). Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, pages 8024–8035.
- [Pei et al., 2020] Pei, H., Wei, B., Chang, K., Lei, Y., and Yang, B. (2020). Geom-gcn: Geometric graph convolutional networks. In *International Conference on Learning Representations*.
- [Peixoto and Emmert-Streib, 2019] Peixoto, T. P. and Emmert-Streib, F. (2019). Network cartography of university websites: A comparative analysis. In *Proceedings of the 30th International Conference on Computer Networks*, pages 1–11.
- [Platonov et al., 2023] Platonov, S. V., Malliaros, F. D., and Michailidis, G. (2023). A critical look at the evaluation of gnns under heterophily: Are we really making progress? *arXiv preprint arXiv:2301.04657*.
- [Rossi et al., 2023] Rossi, E., Charpentier, B., Di Giovanni, F., Frasca, F., Günnemann, S., and Bronstein, M. M. (2023). Edge directionality improves learning on heterophilic graphs. In *Learning on Graphs Conference*, pages 25–1. PMLR.
- [Rozemberczki et al., 2021] Rozemberczki, B., Davies, R., Sarkar, R., and Sutton, C. (2021). Multi-scale attributed node embedding. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, pages 1235–1243.
- [Xu et al., 2018] Xu, K., Hu, W., Leskovec, J., and Jegelka, S. (2018). How powerful are graph neural networks? *arXiv preprint arXiv:1810.00826*.

-
- [Xue et al., 2024] Xue, Y., Jin, Z., and Gao, W. (2024). A data-centric graph neural network for node classification of heterophilic networks. *International Journal of Machine Learning and Cybernetics*, pages 1–11.
- [Yan et al., 2022] Yan, Y., Hashemi, M., Swersky, K., Yang, Y., and Koutra, D. (2022). Two sides of the same coin: Heterophily and oversmoothing in graph convolutional neural networks. In *2022 IEEE International Conference on Data Mining (ICDM)*, pages 1287–1292. IEEE.
- [Yang et al., 2016] Yang, Z., Cohen, W. W., and Salakhutdinov, R. (2016). Revisiting semi-supervised learning with graph embeddings. In *International Conference on Machine Learning*, pages 40–48. PMLR.
- [Ying et al., 2018] Ying, Z., You, J., Morris, C., Ren, X., Hamilton, W., and Leskovec, J. (2018). Hierarchical graph representation learning with differentiable pooling. *Advances in neural information processing systems*, 31.
- [Zhao et al., 2020] Zhao, Q., Ye, Z., Chen, C., and Wang, Y. (2020). Persistence enhanced graph neural network. In *International Conference on Artificial Intelligence and Statistics*, pages 2896–2906. PMLR.
- [Zhu et al., 2021] Zhu, J., Rossi, R. A., Rao, A., Mai, T., Lipka, N., Ahmed, N. K., and Koutra, D. (2021). Graph neural networks with heterophily. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 11168–11176.
- [Zhu et al., 2023] Zhu, J., Yan, Y., Heimann, M., Zhao, L., Akoglu, L., and Koutra, D. (2023). Heterophily and graph neural networks: Past, present and future. *IEEE Data Engineering Bulletin*.
- [Zhu et al., 2020] Zhu, J., Yan, Y., Zhao, L., Heimann, M., Akoglu, L., and Koutra, D. (2020). Beyond homophily in graph neural networks: Current limitations and effective designs. *Advances in Neural Information Processing Systems*, 33:7793–7804.